CAAM 37711 — Homework 1 Solutions (Fall 2026)

Filippo Delzanno

October 14, 2026

Problems 1–3: Handwritten solutions

Filippo Delzanno

Problem Set #1 (CAAM 37711)

(1) a - We let Victor's utility function be the amount of money that he has. Supposing his entire bankroll is d, then the amount Victor can bet on Biden is a number $b \in [0, d]$.

- We can suppose that a market pricing Biden at $\mathbb{P}(\text{Biden wins}) = 0.6 \equiv q$ will sell contracts for $\$q$ paying $1 if Biden wins. With $1, one can buy $\frac{1}{q}$ contracts that pay out $(+\$1 \times \frac{1}{q}) = \frac{1}{q}$. The total payout in this market is thus:

$$\text{profit} \equiv \text{payout} - \text{stake} = \frac{1}{q} - 1 = \frac{1-q}{q} \quad \text{for } q = 0.6$$

Thus, we can develop a utility function for Victor that describes the value of betting $\$b$ on Biden winning, given the probability with which he believes Biden will win. Letting $X \equiv$ Biden wins the election:

$$U(b, X) = \begin{cases} d + \frac{(1-q)}{q} b & ; \text{ if Biden wins, } \mathbb{P}(X) = 0.85 \\ d - b & ; \text{ if Biden loses, } \mathbb{P}(\neg X) = 0.15 \end{cases}$$

- The expected utility under this function for market odds $q = 0.6$ is:

$$\mathbb{E}[U(b,X)] = (0.85)\left[d + \frac{(1-0.6)}{0.6} b\right] + (0.15)[d - b] = d + \frac{5}{12} b$$

Since Victor is a rational agent, he will maximize their expected utility under:

$$\boxed{b^* = \underset{b}{\operatorname{argmax}}\ \mathbb{E}_{X \sim p}[U(b,X)] = \underset{b \in [0,d]}{\operatorname{argmax}} \left(d + \frac{5}{12} b\right) = d,}$$

which is maximized for $b^* = d$, since the expected utility is a strictly increasing function of b. Thus, we have shown that if Victor's utility function is the amount of money that he has, he will bet his entire bankroll on Biden winning.

This kind of linear utility of money does NOT necessarily make sense. The suggestion that one should spend everything they have on a bet with 15% chance of failing will likely be viewed as irrational, since losing everything is a strong detriment than maximizing wins. Similarly, following this rule on any favorable bet will eventually lead to complete ruin, since the probability of winning n repeated bets at $\mathbb{P}(X) = 0.85$ goes like $(0.85)^n \to 0$ for large n. This is a problem for this principle, which suggests you bet everything each time, eventually leading to a complete loss. More broadly, strict adherence to this linear utility principle is not sensible, since it would avoid any decision that incurs a negative expected value. Hedging by buying insurance, for example, is a typical decision by rational agents that would incur negative expected value, contradicting the principle of linear utility of money.

Filippo Delzanno

Problem Set #1 (CAAM 37711)

(1) b - Now, let's suppose that Victor's utility from y is given by utility function:

$$U(y) = \log y$$

- We can write the bet b as a fraction of Victor's wealth $f \in [0,1]$ for convenience, such that his total net worth after the bet is:

$$\text{Net worth} = \begin{cases} d + \frac{1-q}{q} b = d + \frac{1-q}{q} fd = d\left(1 + \frac{1-q}{q} f\right) & ; \text{ if Biden wins} \\ d - b = d - fd = d(1-f) & ; \text{ if Biden loses} \end{cases}$$

- Victor's expected utility under this scheme becomes:

$$\begin{aligned} \mathbb{E}[U(f,X)] &= 0.85 \log\left[d\left(1 + \tfrac{1-q}{q} f\right)\right] + 0.15 \log[d(1-f)] \\ &= 0.85 \log d + 0.85 \log\left[\left(1 + \tfrac{1-q}{q} f\right)\right] + 0.15 \log d + 0.15 \log(1-f) \\ &= \log d + \left\{0.85 \log\left[\left(1 + \tfrac{2}{3} f\right)\right] + 0.15 \log(1-f)\right\}, \end{aligned}$$

where in the last step, we substitute $q = 0.6$ from the betting markets. A rational agent will choose to maximize their expected utility, so we can optimize $\mathbb{E}[U(f,X)]$ to get the fraction f of his total wealth that Victor should commit to the bet:

$$\frac{d\mathbb{E}}{df} = \frac{0.85\left(\frac{2}{3}\right)}{1 + \frac{2}{3} f} - \frac{0.15}{1-f} = 0 \quad \rightarrow \quad \frac{17}{30}\left(1 - f^*\right) = \frac{3}{20}\left(1 + \frac{2}{3} f^*\right)$$

$$\rightarrow \boxed{f^* = \frac{5}{8}}$$

$$\left.\frac{d^2\mathbb{E}}{df^2}\right|_{f=f^*} = \left[-\frac{17}{45(1+\frac{2}{3}f)^2} - \frac{0.15}{(1-f)^2}\right]\Bigg|_{f=\frac{5}{8}} \approx -1.25 < 0$$

$\rightarrow$ f^* is the global maximum on $f \in [0,1]$

$$\rightarrow \left.\frac{d\mathbb{E}}{df}\right|_{f=0} = \frac{17}{30} - \frac{3}{20} = \frac{5}{12}$$

$$\rightarrow \left.\frac{d\mathbb{E}}{df}\right|_{f=1} \rightarrow -\infty \text{ as } f \rightarrow 1$$

Thus, to maximize his expected utility under $U(y) = \log y$, Victor should bet $b = \frac{5}{8} d$ on Biden winning.

c - I probably would not make the bet, on the basis of the uncertainty between the poll numbers and the market odds. Without at least another data point to discriminate whether the spread in probability is a result of a hidden variable or a genuine market mispricing, I would be hesitant to risk 60% of my net worth on a bet that has probability within 15 - 40% of failing. With this uncertainty, I would feel more secure betting 25 - 50% of the amount calculated in part (b) at the expense of smaller gains. If I found additional information that drops the spread in probability of losing and narrows on ± 15% (or less), I would be more confident in applying the bet at full size.

Filippo Delbano

Problem Set #1 (CAAM 37711)

(2) Let's suppose that Y is binary with outcome $Y \in \{0,1\}$. The predictive function f assigns $x \in X$ a number $f(x) \in [0,1]$ that corresponds to the predicted conditional probability $\mathbb{P}[Y=1 \mid X=x]$. From lecture, we know that the cross-entropy loss will be:

$$\mathcal{L}(y,p) = -[y \log p + (1-y)\log(1-p)]$$

- we can denote the TRUE conditional probability by a function $\eta(x) = \mathbb{P}[Y=1 \mid X=x]$. To show that the cross-entropy loss is a proper scoring rule, we must simply show that the risk is minimized when $f^*(x) = \eta(x)$, i.e.:

$$f^* \equiv \underset{f}{\arg\min}\ \mathbb{E}_{X,Y}[\mathcal{L}(Y, f(X))] \quad \text{when } f^*(x) = \eta(x) = \mathbb{P}[Y=1 \mid X=x]$$

Pf | Let's compute the expected loss for a fixed value of x:

$$\mathbb{E}_{X,Y}[\mathcal{L}(Y,f(X))] = \mathbb{E}_X\left[\underbrace{\mathbb{E}[\mathcal{L}(Y,f(X)) \mid X]}\right] \quad \text{[Tower property]}$$

for fixed x, $p = f(x)$ and $\eta(x) = \eta$ such that $Y=1$ with probability η and $Y=0$ with probability $1-\eta$

$$= \mathbb{E}_X\left[-(\eta \log p + (1-\eta)\log(1-p))\right]$$

→ the overall risk is the average of the expected loss at fixed x over the distribution of X

- call this term $g_x(p) \equiv -(\eta \log p + (1-\eta)\log(1-p))$. Since the value of $p = f(x)$ at a fixed point x only affects this function, we should choose the $f(x)$ that minimizes $g_x(p)$ at every x. No other choice of $f(x)$ can do better than this, so this will also be the optimal minimizer in the average $\mathbb{E}_X[g_x(p)]$. We can minimize this function for $0 < \eta < 1$:

$$\frac{dg_x(p)}{dp} = -\frac{\eta}{p} + \frac{1-\eta}{1-p} = 0 \quad \text{[optimization condition]}$$

$$\rightarrow \eta(1-p) = p(1-\eta) \rightarrow p^* = \eta \quad \text{[there is only 1 critical point]}$$

$$\left.\frac{d^2 g_x(p)}{dp^2}\right|_{p^*} = \left[\frac{\eta}{p^2} + \frac{1-\eta}{(1-p)^2}\right]\Bigg|_{p^*=\eta} = \frac{1}{\eta} + \frac{1}{1-\eta} > 0 \quad \forall\ \eta \in (0,1)$$

* at the endpoints $p = 0, 1$: $g_x(p)\big|_{p\to 0} \rightarrow \infty$ AND $g_x(p)\big|_{p\to 1} \rightarrow \infty$

Thus, the critical point $p^* = \eta$ is the unique minimum on $0 < \eta < 1$. Now, we should check $\eta = 0$ and $\eta = 1$, since these are possible values for the true conditional probability:

- if $\eta = 0$, $g_x(p) = -\log(1-p)$, which has its minimum at $p = 0 = \eta$] since $p \in [0,1]$
- if $\eta = 1$, $g_x(p) = -\log(p)$, which has its minimum at $p = 1 = \eta$

Filippo Delzanno

Problem Set #1 (CAAM 37711)

(2) Thus, in every case, the value of p that minimizes $q_x(p)$ is $p = \eta(x)$. This implies that, for every x, the expected loss at that x is uniquely minimized by taking:

$$p^* = f^*(x) = \eta(x)$$

- since we showed that minimizing the expected loss at every x will minimize the implied risk, we can conclude that the prediction function that minimizes the implied risk is

$$f^R(x) = \eta(x) \equiv \mathbb{P}[Y \mid X = x]^{\dagger}$$

† initially, we defined $\eta(x) \equiv \mathbb{P}[Y = 1 \mid X = x]$, but since Y is binary, the entire conditional distribution $\mathbb{P}[Y \mid X = x]$ is specified by knowing $\eta(x)$. So they are interchangeable.

Thus, the cross-entropy loss is a proper scoring rule in the case where Y is binary. ☒

(3) The convergence condition $\sum_t \rho_t^2 < \infty$ that we discussed in class will guarantee that stochastic gradient descent (SGD) will converge to the minimum of the objective function. In a finite-sample setting, we can typically interpret as the empirical risk

$$\hat{R}_n(w) = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}[w; x_i, y_i]$$

as an objective function. For this objective, the minimizer $\hat{w}_n$ is only a noisy estimate of the optimal parameters of the population. This implies that there is some intrinsic noise between the true optimum and $\hat{w}_n$ that cannot be reduced by refining your optimization step. Thus, once your optimization error falls below this threshold, you cannot improve your test performance by increasing the precision of your SGD step.

By instead selecting a constant step size, we can perform a SGD that rapidly converges toward $\hat{w}_n$ and then fluctuates around it in a neighborhood whose size likely scales with the step size ρ and some method-intrinsic vicinity. If we choose ρ such that the neighborhood of fluctuation about $\hat{w}_n$ is smaller than the noise threshold discussed above, we reach the highest accuracy we are capable of much faster than if we were to decay the step size.

Further, we can imagine that a finite-sample setting could host conditions in which setting at the absolutely the optimal parameters would require intuition about the training set that does not reflect the real trend that produced the data. We would have to spend considerable resources to derive parameters that are unnecessary (or worse, obfuscating). Thus, violating the formal convergence criteria can be worthwhile in practice in order to reach an approximately acceptable solution very quickly.

All code for this assignment is in the GitHub repository `https://github.com/fdelzanno/CAAM-37711`, under the `homework-1/` directory. See `README.md` for setup and run instructions.

Problem 4

The problem recommends that we fit different regression models from the `sklearn` package. Following that recommendation, we will use the following models:

- **Linear Regression**: fits a traditional linear model with corresponding weights by minimizing the squared error.
- **Support Vector Regression (SVR)**: SVR fits a function such that as many training points as possible lie within a "tube" in feature space of width $\pm\epsilon$ around it. The fit ignores errors inside the tube and penalizes errors outside in proportion to their size. We pass a regularization parameter C that sets the tradeoff between fitting data closely and keeping the resulting function smooth (i.e., a larger C penalizes points outside the tube more heavily). To capture nonlinear relationships, we use a radial basis function (RBF) kernel $K(x, x') = \exp\big(-\gamma\|x - x'\|^2\big)$. This measures the similarity between two points as a decreasing function of their distance such that the resulting prediction is a weighted sum of smooth, Gaussian "bumps" centered on a subset of the training points (called the support vectors). In this way, each prediction is mainly influenced by nearby training data. Since the kernel depends on distances in feature space, we standardize all four features to zero mean and unit variance before fitting to avoid a single feature dominating.
- **Decision Tree**: this regressor predicts the target by splitting the data into smaller groups sorted by particular characteristics of the features. At each step, the algorithm chooses the feature and threshold that best separate the data (i.e., the split that most reduces the squared error between the observations and the mean of their group). Since this splitting continues until a stopping rule is reached, the final groups (called leaves) will each predict the average **ale** of the training points it contains. This makes the fitted model a step function with abrupt jumps at the splitting thresholds. Since an unrestricted tree could continue to split until every leaf holds a single training point, we artificially limit the tree to stop at a depth of 4 (giving at most 16 leaves).
- **Neural Network**: predicts the target by passing the inputs through a series of layers made of simple computational units. Each unit takes a weighted sum of inputs, adds a constant (the bias), and applies a nonlinear activation function. In this case, we apply the rectified linear unit (ReLU), which outputs the value unchanged if it is positive and zero otherwise. Stacking layers of these units lets the network build up complex, nonlinear relationships from the pieces, creating an output that is a weighted combination of the last hidden layer's values. For this example, we will use two hidden layers of 32 units each. The weights and biases will start at random values and are adjusted iteratively by gradient descent to minimize the squared error between predictions and observed values. We allow up to 5,000 training iterations since we are dealing with a small dataset.

Since the SVR and neural network models are sensitive to the scale of their input features, a feature with a large numeric range can dominate the fit. To prevent this, we apply scikit-learn's `StandardScaler` to transform each feature into having zero mean and unit variance. Then, we combine the scaler and the model into a single `Pipeline` so that the means/standard deviations are computed from the training set only, and the same transformation is applied to the test data at

prediction time. This should prevent information from the test set from leaking into the training data. Since linear regression and decision tree are unaffected by feature scaling, we do not apply this procedure to them.

To assess whether the computed residuals are approximately normal, we will follow the problem's suggestion: combine a QQ plot with a `scipy` normal test. The quantile-quantile (QQ) plot sorts the residuals from smallest to largest and plots each one against the value we would expect at that position if the data came from a normal distribution. These give us a visual check on the normality of the residuals, since normal residuals will lie on a straight line on the QQ plot. The formal test is the D'Agostino–Pearson test (`scipy.stats.normaltest`), which measures the residuals' skewness and kurtosis, compares each to the value expected under normality, and combines the two into a single statistic. The null hypothesis is that the residuals are drawn from a normal distribution, so a p-value below 0.05 is taken as evidence **against** normality. A large p-value means that the test finds no significant departure from normality.

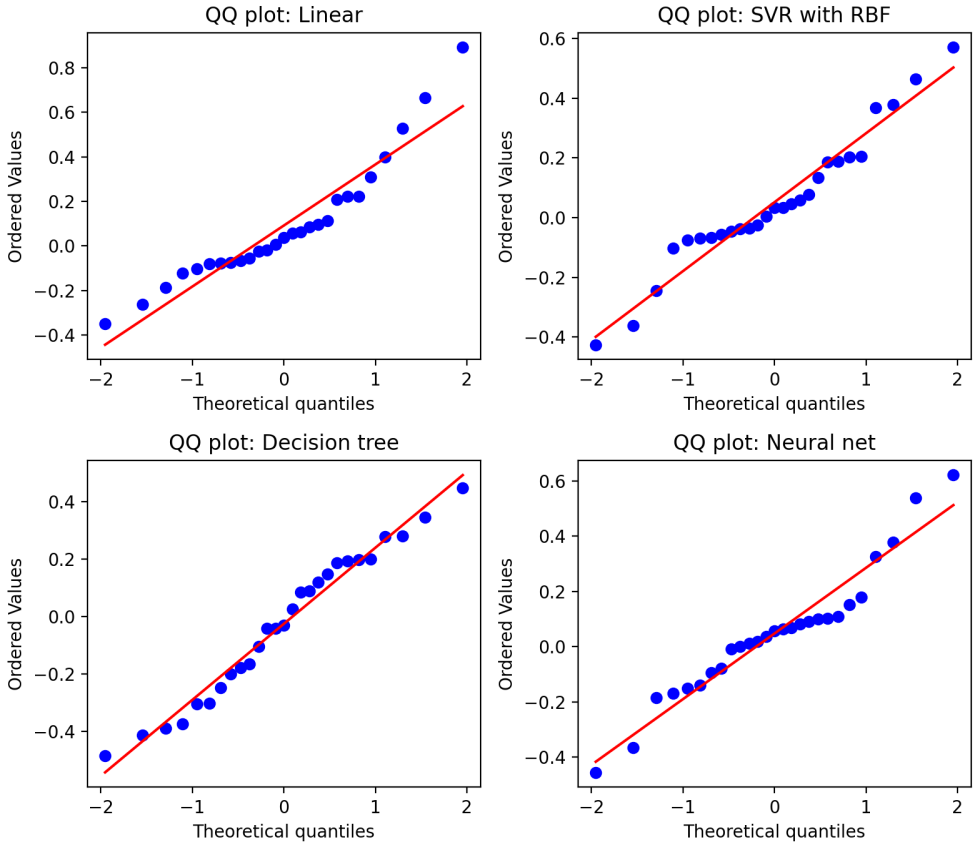


Figure 1: QQ plots of test residuals (points) against the normal distribution (red line).

We fit four regression models (linear regression, SVR with an RBF kernel, a decision tree, and a neural network) to predict ALE from four features: anchor ratio, transmission range, node density, and iterations. We tested whether each model's residuals on a test set are approximately normal, and found that all three nonlinear models outperformed linear regression. Further, the D'Agostino–Pearson test rejected normality only for the linear model and failed to reject normality at the 0.05 significance level for the other three. This indicates that the relationship between the features and

Model	Test RMSE	Statistic	p-value	Reject at 0.05?
Linear	0.287	9.660	0.008	Yes
SVR with RBF	0.228	1.024	0.599	No
Decision tree	0.253	2.674	0.263	No
Neural net	0.236	2.616	0.270	No

Table 1: Test RMSE and normality test of the test-set residuals ($n = 27$).

ALE is likely nonlinear and cannot be fully captured by a simple family of linear models.

These results suggest that the non-normality of the linear model's residuals is likely a symptom of model misspecification (as opposed to non-Gaussian noise in the data). Since linear models cannot capture the nonlinear structure in ALE, their systematic errors must remain in the residuals and distort their distribution. Indeed, the QQ plot of the linear model's residuals is convex, with a short lower tail and a long upper tail, indicative of right skew driven by a small number of large positive residuals. More flexible models can capture this structure, leaving residuals consistent with approximately normal noise.

Overall, the results are consistent with the common justification for squared error loss cited in the problem statement: that the fitted residuals are approximately normally distributed. This result comes with the caveat, however, that the model must be flexible enough to capture the underlying relationship of the data, and should be interpreted in the context of this small dataset. With only 27 test residuals per model, the normality test has limited power, and failing to reject normality in this test is not the same as showing that the residuals are truly normal. We have shown that approximate normality is an appropriate justification, but would need to repeat this procedure on a larger test sample to establish normality in the residuals with more confidence.

Problem 5

Classification (Breast Cancer Wisconsin)

As requested by the problem, we perform an 80/20 train-test split on the data. We collect a baseline result using scikit-learn's `LogisticRegression` on the training data and measure prediction accuracy on the test sample. Then, we optimize two models: (a) logistic regression $f(x) = \sigma(w^\top x + b)$ and (b) probit regression $f(x) = \Phi(w^\top x + b)$ by minimizing a binary cross-entropy loss function (truncated to $[10^{-7}, 1 - 10^{-7}]$ to avoid numerical instabilities). We noticed that the training loss drops quickly in the first few epochs and then keeps decreasing slowly. Therefore, we report results for 200 epochs, at which point the loss has largely flattened, and a learning rate of 0.1.

Method	Test accuracy	Correct
sklearn baseline	96.49%	110/114
SGD (logistic model)	97.37%	111/114
SGD (probit model)	95.61%	109/114

Table 2: Test accuracy on the Breast Cancer test set.

The SGD logistic model slightly exceeds the baseline (by 1 test observation), while the SGD probit model is slightly below it (by 1 test observation). Since the gaps are single test observations out of

114, they are within the variability that could be produced by varying the learning rate, number of epochs, or using different train/test splits. Note that the baseline also applies scikit-learn's default ℓ_2 regularization, which the SGD models do not use (they are unregularized). This may account for part of the difference. Overall, the two model functions give very similar classifiers, as expected, since σ and Φ are both smooth S-shaped maps from the real line to $(0, 1)$.

Regression (ALE)

As requested by the problem, we perform an 80/20 train-test split on the data. We collect a baseline result by applying scikit-learn's `LinearRegression` on the training data and computing the mean absolute error (MAE) on the test sample. Then, we optimize a Linear Regression model on both (a) the mean squared error loss and (b) the Student-t negative log-likelihood with $\nu = 5$ degrees of freedom and compare the mean absolute error on the test data using parameters learned from each loss function with the baseline result. By testing combinations of parameters, we determined that using 200 training epochs and a learning rate of 0.05 was enough to converge the model training loss to a sufficient plateau.

Method	Test MAE
sklearn LinearRegression	0.1693
SGD, MSE loss	0.1700
SGD, Student-t NLL ($\nu = 5$)	0.1692

Table 3: MAE on the ALE test set ($n = 22$).

Both SGD-trained models achieve a test MAE essentially identical to the baseline: the MSE-trained model is slightly worse, and the Student-t-trained model is slightly better. These gaps are less than 10^{-3} on only 22 test observations, so they are well within the variability that could be produced by changing the number of epochs and learning rate, or using a different train/test split. From this data alone, we cannot conclude that either loss is better. Since the MSE-trained SGD model minimizes the same loss function as `LinearRegression`, it should approach the least-squares solution. This is consistent with what we see for the MSE, and the small difference likely reflects the finite training we use for SGD or batch sampling noise. As noted in the problem statement, the Student-t distribution has heavier tails, so minimizing its loss function will differ from squared error mainly for large residuals. When we looked at the residuals of the linear model in Problem 4 (Fig. 1), we saw that they lie between about -0.35 and 0.9, with most within ± 0.4. Thus, we do not expect the heavy-tailed loss to change the fit much, which is consistent with our observation that its test MAE is nearly identical to that of the MSE-trained model.